

16-bit Single-Cycle CPU — Instruction Set Reference

Custom CPU project built in Verilog

This document describes a custom-designed 16-bit, single-cycle CPU: a 16-bit instruction word, a 16-register file, a shared ALU handling arithmetic, logic, shift, and comparison operations, and a small branch/flag mechanism for conditional control flow. Every instruction completes fetch, decode, execute, and writeback within one clock cycle.

Architecture at a Glance

- **Program Counter** — holds the current instruction address; increments each cycle or loads a branch target.
- **Instruction Memory** — 256 x 16-bit ROM, addressed by the PC.
- **Main Decoder** — turns the opcode and mode_bits into control signals (reg_write, alu_src, mem_read, mem_write, mem_to_reg, branch, alu_op, hlt).
- **Register File** — 16 general-purpose 16-bit registers (R0-R15), two independent read ports and one write port.
- **ALU** — performs ADD, SUB, AND, OR, NOT, XOR, MUL, MULS, all four shift/rotate types, and both signed and unsigned comparison.
- **Flag Register** — latches zero/carry/sign/overflow/eq/gt/lt only on ADD, SUB, CMP, or CMPS, so branches always see the result of the most recent flag-setting instruction.
- **Main Memory** — 256 x 16-bit data RAM for load/store instructions.
- **Branch Logic** — evaluates the latched flags against the branch's condition code to decide whether to redirect the PC.

1. Instruction Word Format

Every instruction is 16 bits, split into four 4-bit fields:

Bits	Field	Description
[15:12]	Opcode	Selects the instruction group, one of 16 (Section 2).
[11:8]	mode_bits, or destination register (Rd)	MOV / Branch-Compare / Shift groups: mode_bits, selecting the specific sub-instruction (Sections 3-5). All other groups (ADD, SUB, AND, OR, NOT, XOR, MUL, MULS): destination register Rd.
[7:4]	Ra, or destination register (Rd) for MOV/Shift	ALU-type groups: source register Ra. MOV and Shift groups: destination register — the result overwrites this register directly.
[3:0]	Rb, 4-bit immediate, or branch target (low nibble)	ALU-type groups: source register Rb. When alu_src selects an immediate: a 4-bit unsigned immediate, range 0-15 (zero-extended). Branch/Compare group: combines with [7:4] is NOT used here — see Section 4, the full byte [7:0] carries the branch target.

2. Top-Level Opcode Map

Opcode	Instruction
0000	RST
0001	MOV — see Section 3
0010	Branch / Compare — see Section 4
0011	Shift — see Section 5
0100	ADD Rd, Ra, Rb
0101	ADD Rd, Ra, #imm
0110	SUB Rd, Ra, Rb
0111	SUB Rd, Ra, #imm
1000	AND
1001	OR
1010	NOT
1011	XOR
1100	MUL
1101	MULS
1110	NOP
1111	HLT

3. MOV Sub-modes (mode_bits, opcode = 0001)

mode_bits	Instruction	Description
0000	MOV Rd, Rs	Register-to-register copy.
0011	MOV Rd, #imm	Load a 4-bit unsigned immediate into Rd.
0100	MOV [Ra], Rb	Store: memory address = Ra, data written = Rb.
1000	MOV Rd, [Ra]	Load: memory address = Ra, result written to Rd.

No further MOV sub-modes are currently defined; the remaining mode_bits values are unused.

4. Branch / Compare Sub-modes (mode_bits, opcode = 0010)

mode_bits	Mnemonic	Condition checked
0000	B	Unconditional
0001	BZ	zero = 1
0010	BNZ	zero = 0
0011	BC	carry = 1
0100	BNC	carry = 0
0101	BV	overflow = 1
0110	BNV	overflow = 0
0111	—	unused
1000	BEQ	eq = 1
1001	BNE	eq = 0
1010	BGT	gt = 1
1011	BLT	lt = 1
1100	BGE	gt = 1 OR eq = 1
1101	BLE	lt = 1 OR eq = 1
1110	CMPS	Signed compare — sets alu_op, branch forced to 0
1111	CMP	Unsigned compare — sets alu_op, branch forced to 0

The branch target address occupies the full low byte of the instruction, bits [7:0] — reachable range is addresses 0-255, matching the 256-word instruction memory. Flags are read from the dedicated flag register (latched on ADD, SUB, CMP, CMPS), not directly from the ALU's live output.

5. Shift Sub-modes (mode_bits, opcode = 0011)

mode_bits	Instruction	Notes
0000	Rshift Ra, Rb	SHR
0001	Rshift Ra, #imm	SHR, immediate operand
0010	Lshift Ra, Rb	SHL
0011	Lshift Ra, #imm	SHL, immediate operand
0100	Arith-right Ra, Rb	ASHR
0101	Arith-right Ra, #imm	ASHR, immediate operand
0110	ROR Ra, Rb	Rotate right
0111	ROR Ra, #imm	Rotate right, immediate operand
1000	ROL Ra, Rb	Rotate left
1001	ROL Ra, #imm	Rotate left, immediate operand

Destination register is bits [7:4] — same field as the source register; the result overwrites the source.

6. ALU Operation Encoding (alu_op)

alu_op	Operation
0	ADD
1	SUB
2	MUL (unsigned)
3	MULS (signed / Booth)
4	ASHR
5	SHL
6	SHR
7	ROL
8	ROR
9	CMP (unsigned)
10	CMPS (signed)
11	AND
12	OR
13	NOT
14	XOR
15	MOV pass-through

This numbering is confirmed and matches main_decoder.v. CMP/CMPS use a special-cased zero/eq flag computation rather than the generic (result == 0) check, since their ALU output holds the comparison bit itself rather than an arithmetic result.

7. Flags

Flag	Definition
Z (zero)	Set when the ALU result is zero (ADD/SUB). For CMP/CMPS, set directly from the eq bit.
C (carry)	Carry-out from the adder/subtractor, ADD/SUB only. Verified against a hand-traced test case.
S (sign)	MSB of the ALU result.
V (overflow)	Signed overflow from operand sign bits vs. result sign bit, ADD/SUB only.
eq / gt / lt	From the unsigned or signed comparator depending on CMP vs. CMPS. Meaningful once latched into the flag register.

8. Design Notes

- immediates are 4 bits, zero-extended (unsigned), range 0-15 — not sign-extended.
- branch targets use the full byte, instruction bits [7:0], range 0-255.
- alu_op encoding in Section 6 is confirmed and matches the decoder.
- SUB carry/borrow polarity has been verified against a hand-traced test case.

Open for future work:

- MOV sub-modes beyond register-copy, #imm, load, and store are not yet defined.